



A.D. 1308
unipg
DIPARTIMENTO
DI INGEGNERIA

UNIVERSITÀ DEGLI STUDI DI PERUGIA
DIPARTIMENTO DI INGEGNERIA

Tesina Finale di
Algoritmi e Strutture Dati

Corso di Laurea in Ingegneria Informatica ed Elettronica — A.A. 2025–2026

**Titolo
molto
lungo**

Docente
Prof. Emilio Di Giacomo

Studente
Romeo Miscio
Matricola: 364672

Ultimo aggiornamento: 18 giugno 2026

Indice

1	Il problema della generazione di labirinti	1
2	Modellazione formale tramite Teoria dei Grafi	1
3	Obiettivi del progetto	2
4	Algoritmi implementati	2
4.1	Randomized Depth-First Search (Recursive Backtracker)	2
4.1.1	Scelte implementative e gestione dello Stack	2
4.1.2	Analisi della Complessità Teorica	3
4.2	Algoritmo di Kruskal Randomizzato	3
4.2.1	La struttura dati Disjoint-Set (Union-Find)	3
4.2.2	Analisi della Complessità Teorica	4
4.3	Algoritmo di Prim Modificato	4
4.3.1	Scelte implementative sulla Frontiera	4
4.3.2	Analisi della Complessità Teorica	4
4.4	Algoritmi basati su Random Walk	5
4.4.1	Algoritmo di Aldous-Broder	5
4.4.2	Algoritmo di Wilson	5
4.4.3	Analisi della Complessità Teorica	6
4.5	Algoritmi geometrici e a partizione spaziale	6
4.5.1	Algoritmo di Divisione Ricorsiva	6
4.5.2	Analisi della Complessità Teorica	7
4.5.3	Algoritmo di Eller	7
4.5.4	Analisi della Complessità Teorica	7
5	Esperimenti e Analisi Dati	8
5.1	Setup Sperimentale	8
5.1.1	Metodologia di Profilazione e Campionamento	8
5.2	Risultati Sperimentali Globali	9
5.3	Discussione Critica dei Risultati	9
5.3.1	Il dominio degli approcci Top-Down: Divisione Ricorsiva	9
5.3.2	L'efficienza dei Graph Traversal e delle Strutture Dati: DFS, Kruskal ed Eller	10
5.3.3	Analisi del collasso asintotico nei Random Walk: Wilson vs Aldous-Broder	10
5.3.4	Andamento dell'algoritmo di Prim	11

1 Il problema della generazione di labirinti

Il problema della generazione di labirinti perfetti consiste nel ricavare una struttura planare complessa dotata di un'unica interconnessione spaziale priva di interruzioni (regioni isolate) e priva di percorsi alternativi chiusi (cicli) [1].

In ambito informatico e computazionale, un labirinto non rappresenta semplicemente un elemento ludico, ma si configura come un eccellente banco di prova per lo studio della topologia strutturale, della teoria dei grafi e dell'efficienza algoritmica nell'esplorazione spaziale. Trova inoltre ampie applicazioni nella generazione procedurale di contenuti (PCG) nei videogiochi, nella pianificazione di percorsi per la robotica mobile e nella simulazione di reti di micro-canali biologici o idraulici.

2 Modellazione formale tramite Teoria dei Grafi

Il modo formalmente più rigoroso per descrivere la topologia di un labirinto adotta le definizioni della teoria dei grafi. Una griglia bidimensionale predeterminata (sia essa rettangolare, esagonale o circolare) può essere mappata direttamente su un grafo non diretto e pesato $G = (V, E)$, in cui:

- L'insieme dei vertici V rappresenta l'insieme delle celle discrete che compongono lo spazio calpestabile del labirinto, con $|V| = n$.
- L'insieme degli archi E rappresenta l'insieme di tutti i passaggi originariamente potenziali (ovvero l'assenza di un muro divisorio) tra celle geometricamente adiacenti.

Generare un labirinto strutturalmente valido significa individuare un sottografo accatato $G' = (V, E')$ tale che $G' \subseteq G$, che rispetti due vincoli stringenti:

1. **Connettività:** Il sottografo G' deve essere strettamente connesso. Se G' non fosse connesso, esisterebbero regioni isolate e strutturalmente irraggiungibili all'interno della griglia, invalidando l'integrità del labirinto.
2. **Alinearità (Assenza di cicli):** Il sottografo G' deve essere aciclico. La presenza di un ciclo implicherebbe l'esistenza di cammini multipli per connettere due nodi qualsiasi, riducendo la complessità intrinseca del labirinto e violando la definizione classica di "labirinto perfetto".

Dalle proprietà fondamentali della teoria dei grafi, un sottografo di G che contiene tutti i vertici V , che sia connesso e che è privo di cicli, per definizione è un **Albero di Copertura** (o *Spanning Tree*). Di conseguenza, l'intero problema della generazione procedurale di labirinti si riduce all'estrazione di uno **Spanning Tree casuale** partendo dal grafo iniziale della griglia. I muri del labirinto finito saranno costituiti esattamente dal complemento degli archi selezionati, ovvero dall'insieme degli archi esclusi $E \setminus E'$.

3 Obiettivi del progetto

L'obiettivo principale di questo progetto è implementare, analizzare e confrontare sperimentalmente le prestazioni e le caratteristiche topologiche delle principali famiglie di algoritmi per l'estrazione di Spanning Tree casuali.

Nello specifico, il software sviluppato in linguaggio Java mira a ricoprire l'intero spettro algoritmico descritto nella pagina di Wikipedia [1], dividendoli per tipologia di funzionamento:

- **Algoritmi basati su Tree/Graph Traversal:** Randomized Depth-First Search (Recursive Backtracker);
- **Algoritmi basati su Minimum Spanning Tree adattati:** Algoritmo di Kruskal randomizzato e Algoritmo di Prim modificato.
- **Algoritmi basati su Random Walk:** Algoritmo di Aldous-Broder e Algoritmo di Wilson.
- **Algoritmi geometrici o a divisione spaziale:** Algoritmo di Divisione Ricorsiva ed Eller's Algorithm.

Oltre all'analisi prestazionale pura asintotica e di wall-clock, il progetto si propone di integrare un motore grafico per la visualizzazione dinamica passo-passo della crescita dell'albero, controllato da un'euristica di temporizzazione per garantire la fluidità dell'animazione a prescindere dal dominio di calcolo impostato.

4 Algoritmi implementati

In questo capitolo vengono analizzati in dettaglio tutti e sette gli algoritmi sviluppati per la generazione di labirinti perfetti. Ciascun approccio interpreta il problema dell'estrazione di uno Spanning Tree casuale sfruttando strutture dati e paradigmi logici differenti.

4.1 Randomized Depth-First Search (Recursive Backtracker)

L'algoritmo basato sulla ricerca in profondità (DFS) modificata è un approccio classico di tipo *backtracking*. Esso tende a generare labirinti caratterizzati da un basso numero di diramazioni corte e da corridoi marcatamente lunghi e tortuosi (frequenti vicoli ciechi profondi), proprietà topologica nota in letteratura come "alto fattore di fiumi".

4.1.1 Scelte implementative e gestione dello Stack

La formulazione puramente ricorsiva dell'algoritmo DFS soffre intrinsecamente del limite di allocazione dello stack di memoria della Java Virtual Machine (*StackOverflowError*) quando la dimensione della griglia $n = |V|$ scala linearmente. Per ovviare a questa limitazione hardware, si è optato per una **vettorializzazione esplicita dello stack** mediante la struttura dati `java.util.Stack`.

Il comportamento del singolo passo operativo, fondamentale per agganciare il ciclo di animazione della veste grafica senza bloccare il thread di rendering, è regolato dalla seguente logica a stati stabili:

1. Viene esaminata la cella in cima allo stack senza estrarla (`peek()`).
2. Si interroga l'intorno geometrico della cella per estrarre l'insieme dei vicini non ancora visitati.
3. Se l'insieme è non vuoto, viene selezionato casualmente un vicino, viene rimosso il muro divisorio tra le due celle, la nuova cella viene marcata come visitata e inserita nello stack (`push()`).
4. Se non vi sono vicini validi (vicolo cieco), si effettua il ritorno a ritroso rimuovendo la cella corrente dallo stack (`pop()`).

4.1.2 Analisi della Complessità Teorica

In una griglia regolare planare, il grado massimo di ogni vertice è limitato superiormente da una costante ($\deg(v) \leq 4$). Di conseguenza, il numero totale di archi è lineare rispetto al numero dei nodi, ovvero $|E| = \mathcal{O}(V)$. Poiché ogni cella viene inserita ed estratta dallo stack esattamente una volta, il ciclo interno esegue al più $2|V|$ iterazioni. L'accesso e la modifica delle proprietà della cella (muri e flag di visita) avvengono in tempo costante $\mathcal{O}(1)$. La complessità temporale asintotica dell'algoritmo è pertanto strettamente lineare:

$$\mathcal{O}(V)$$

Lo spazio aggiuntivo occupato dallo stack nel caso pessimo (un unico corridoio lineare senza diramazioni) coincide con l'altezza massima dell'albero, richiedendo una complessità spaziale pari a $\mathcal{O}(V)$.

4.2 Algoritmo di Kruskal Randomizzato

L'adattamento dell'algoritmo di Kruskal per la generazione di labirinti opera trattando tutti i muri divisorii della griglia come un insieme di archi non pesati. Al posto di minimizzare il costo totale ordinando gli archi, l'algoritmo estrae un arco in modo puramente casuale a ogni iterazione, fondendo progressivamente i sotto-alberi isolati.

4.2.1 La struttura dati Disjoint-Set (Union-Find)

Per controllare in tempo quasi-costante se l'abbattimento di un muro introduce un ciclo indesiderato, è stata implementata da zero una foresta di insiemi disgiunti (**Disjoint-Set**) dotata di due ottimizzazioni fondamentali:

- **Unione per Rango (Union by Rank):** Attacca sempre l'albero con altezza minore sotto la radice dell'albero con altezza maggiore, limitando la crescita logaritmica delle strutture.
- **Compressione dei Cammini (Path Compression):** Durante la fase di ricerca (`find`), ogni nodo visitato viene agganciato direttamente alla radice dell'insieme, appiattendolo la struttura per le invocazioni successive.

4.2.2 Analisi della Complessità Teorica

Inizialmente la lista contiene la totalità degli archi possibili della griglia, pari a $|E| \approx 2|V|$. Il mescolamento iniziale (`Collections.shuffle`) richiede tempo lineare $\mathcal{O}(E)$. A ogni passo estrattivo, l'algoritmo esegue due operazioni di `find` e al più una operazione di `union`. Sotto le ipotesi di compressione dei cammini e unione per rango, la complessità ammortizzata di ciascuna operazione è limitata superiormente dalla funzione inversa di una crescita esponenziale raddoppiata, espressa mediante la funzione di iterazione di Ackermann $\alpha(n)$. La complessità temporale complessiva per elaborare tutti gli archi è:

$$\mathcal{O}(E \cdot \alpha(V))$$

Poiché nella nostra griglia $|E| = \mathcal{O}(V)$ e la funzione α assume un valore pratico minore di 5 per qualsiasi input realistico, l'algoritmo opera in tempo di *wall-clock* quasi-lineare. Lo spazio aggiuntivo per mantenere i vettori dei padri e dei ranghi è pari a $\mathcal{O}(V)$.

4.3 Algoritmo di Prim Modificato

L'algoritmo di Prim si sviluppa espandendo radialmente un unico sotto-albero a partire da una cella di innesco casuale. Mantiene costantemente traccia di una struttura di “frontiera”, definita come l'insieme di tutti i muri che separano una cella già inclusa nel labirinto da una cella non ancora esplorata.

4.3.1 Scelte implementative sulla Frontiera

Nell'implementazione didattica standard dell'algoritmo di Prim (utilizzato per la ricerca del Minimum Spanning Tree classico), la frontiera viene memorizzata in una coda a priorità (*Min-Heap*) basata sul peso degli archi. Nel contesto della generazione procedurale casuale, l'assenza di pesi deterministici ci permette di sostituire lo heap con una lista dinamica indicizzata (`java.util.ArrayList`).

La rimozione di un elemento casuale da una lista basata su array richiede lo spostamento degli elementi successivi, operazione intrinsecamente inefficiente se eseguita al centro della struttura ($\mathcal{O}(N)$). Per mantenere il costo microscopico nell'interfaccia a passi, si memorizzano i riferimenti strutturali espliciti dei muri (`PrimWall`) e si effettua un campionamento ad accesso diretto.

4.3.2 Analisi della Complessità Teorica

Ogni cella viene visitata una sola volta e aggiunge al più 4 muri alla lista di frontiera. Di conseguenza, il numero massimo di elementi immessi nella lista è proporzionale a $4|V|$. L'estrazione casuale di un indice da una struttura `ArrayList` avviene in tempo costante $\mathcal{O}(1)$, ma la rimozione di un elemento richiede nel caso pessimo uno slittamento lineare di memoria pari alla dimensione istantanea della frontiera stessa, che nel regime denso di esplorazione può contenere fino a $\mathcal{O}(V)$ elementi. La complessità temporale complessiva su griglia si attesta quindi a:

$$\mathcal{O}(V^2)$$

Dal punto di vista spaziale, la frontiera immagazzina al più il perimetro dinamico di crescita dell'albero, occupando in memoria uno spazio delimitato superiormente da $\mathcal{O}(V)$.

4.4 Algoritmi basati su Random Walk

La generazione di un labirinto tramite passeggiate casuali (*Random Walk*) si fonda su principi profondi della teoria delle probabilità e dei processi markoviani. Entrambi gli algoritmi descritti di seguito condividono la proprietà matematica di estrarre un albero di copertura da una distribuzione perfettamente uniforme (*Uniform Spanning Tree*), garantendo che ogni possibile labirinto perfetto compatibile con la griglia iniziale abbia l'esatta stessa probabilità di essere generato.

4.4.1 Algoritmo di Aldous-Broder

L'algoritmo di Aldous-Broder adotta un meccanismo a memoria locale elementare. Partendo da un vertice iniziale arbitrario marcato come visitato, l'algoritmo sceglie a ogni ciclo un vicino geometrico in modo uniforme e casuale. Se il vicino selezionato non è mai stato visitato in precedenza, il muro divisorio tra le due celle viene rimosso e il nuovo nodo viene annesso all'albero. Se il vicino è invece già stato visitato, la passeggiata si sposta semplicemente su di esso senza alterare la struttura dei muri.

L'algoritmo è strutturalmente inefficiente a causa del fenomeno probabilistico noto come *Coupon Collector's Problem* (Problema del collezionista di figurine). Nelle fasi iniziali e intermedie, la scoperta di nuove celle avviene con frequenza elevata. Tuttavia, quando la quasi totalità della griglia è stata annessa, la passeggiata casuale spende la maggior parte dei cicli di clock muovendosi all'interno di regioni già esplorate, nel tentativo stocastico di intersecare gli ultimi nodi rimasti isolati.

Per questa ragione, la complessità temporale asintotica nel caso pessimo non è limitata superiormente da una funzione polinomiale fissa, ma dipende dal tempo di copertura (*cover time*) del grafo griglia, attestandosi mediamente a:

$$\mathcal{O}(V^2)$$

con picchi degeneri che surclassano tale limite su topologie sbilanciate. Lo spazio aggiuntivo richiesto è minimale ed equivalente a $\mathcal{O}(1)$ se escluso il dominio della griglia stessa.

4.4.2 Algoritmo di Wilson

L'algoritmo di Wilson ottimizza drasticamente le inefficienze della passeggiata casuale pura introducendo il concetto di **Loop-Erased Random Walk** (Passeggiata casuale con rimozione dei cicli). La logica separa nettamente lo spazio calpestabile in due stati temporanei: le celle incluse nell'albero consolidato e le celle non ancora esplorate.

La procedura operativa è così ripartita:

1. Viene selezionata una cella casuale originaria e inserita permanentemente nell'albero di copertura.
2. Si sceglie una cella arbitraria al di fuori dell'albero e si avvia una passeggiata casuale standard.

3. Durante il cammino, se la traiettoria interseca un nodo già appartenente alla passeggiata corrente, si è in presenza di un ciclo (*loop*). L'algoritmo tronca immediatamente la struttura eliminando tutti i passi logici compiuti all'interno del ciclo, prevenendo la formazione di cammini multipli.
4. Quando la passeggiata casuale interseca una cella appartenente all'albero consolidato, il cammino corrente viene interrotto: tutti i muri lungo la traiettoria purificata dai cicli vengono abbattuti e l'intero percorso si fonde istantaneamente con l'albero.

4.4.3 Analisi della Complessità Teorica

Sebbene una singola passeggiata possa teoricamente durare a lungo prima di intersecare l'albero, la rimozione sistematica dei cicli fa sì che il tempo necessario per connettere un blocco di nodi dipenda dal tempo di transito medio e non dal tempo di copertura totale del grafo. La complessità temporale media sui grafi griglia bidimensionali regolari si attesta a:

$$\mathcal{O}(V^{1.2})$$

Questo rende l'algoritmo di Wilson asintoticamente molto più efficiente di Aldous-Broder e di Prim, offrendo al contempo la medesima distribuzione perfettamente uniforme del labirinto finale. Dal punto di vista spaziale, la lista dinamica deputata a memorizzare i nodi della passeggiata corrente richiede nel caso pessimo un'occupazione di memoria pari a $\mathcal{O}(V)$.

4.5 Algoritmi geometrici e a partizione spaziale

A differenza degli approcci basati sui grafi che operano a livello locale di vertice o arco, gli algoritmi geometrici sfruttano la regolarità strutturale della griglia per partizionare lo spazio o per calcolare l'albero di copertura secondo vincoli di riga globali.

4.5.1 Algoritmo di Divisione Ricorsiva

L'algoritmo di Divisione Ricorsiva adotta una filosofia progettuale di tipo *top-down* guidata dal paradigma del *divide et impera*. Il processo si avvia considerando la griglia come un'unica macro-regione interamente priva di barriere interne.

L'esecuzione del singolo passo d'avanzamento si articola sulle seguenti sotto-fasi:

1. Viene estratta una regione geometrica dal buffer di elaborazione esplicito. Se le sue dimensioni in pixel logici di cella sono inferiori alla soglia critica di divisione ($w < 2$ o $h < 2$), la regione viene considerata atomica e consolidata.
2. Si valuta l'orientamento ottimale del taglio basandosi sul rapporto d'aspetto della sotto-matrice: se l'altezza supera la larghezza il taglio sarà preferenzialmente orizzontale, viceversa sarà verticale.
3. Viene eretto un muro continuo che seziona longitudinalmente la regione a una coordinata casuale. Successivamente, viene selezionata in modo uniforme una singola cella lungo la barriera appena eretta in cui praticare un'apertura (foro di transito).

4. La separazione genera due nuove sotto-regioni indipendenti, le quali vengono inserite nello stack per essere processate nelle iterazioni successive.

4.5.2 Analisi della Complessità Teorica

La scomposizione dello spazio ricalca in modo duale la struttura di un albero binario di decisione. A ogni livello della ricorsione strutturale, l'algoritmo esegue un numero di operazioni lineare rispetto al perimetro della sotto-regione da murare. Esprimendo la ricorrenza per una griglia quadrata di lato N (dove il numero totale di celle è $|V| = N^2$), si ottiene la relazione di bilancio energetico:

$$T(N^2) = 2T\left(\frac{N^2}{2}\right) + \mathcal{O}(N)$$

Risolvendo mediante l'applicazione del Teorema Master, si ricava una complessità temporale asintotica pari a:

$$\mathcal{O}(V \log V)$$

Lo spazio di memoria ausiliario necessario per alloggiare i record geometrici delle regioni all'interno dello stack è limitato dall'altezza massima dell'albero di partizione, che si attesta a $\mathcal{O}(\log V)$.

4.5.3 Algoritmo di Eller

L'algoritmo di Eller rappresenta uno dei vertici più efficienti della generazione procedurale. Esso risolve il problema calcolando l'albero di copertura riga per riga in un unico passaggio sequenziale. Ciascuna cella della riga corrente viene associata a un identificativo numerico di set, il quale mappa in tempo reale la connettività globale del labirinto calcolato fino a quel momento.

La logica si sviluppa attraverso due macro-fasi per ogni riga:

1. **Fase Orizzontale:** L'algoritmo esamina ogni coppia di celle adiacenti sulla riga. Se appartengono a set diversi, decide in modo stocastico se abbattere il muro che le separa. In caso di abbattimento, i due set vengono fusi, assegnando a tutti i loro membri il medesimo identificativo. All'ultima riga del labirinto, questa unione viene forzata per ogni coppia avente set disgiunti, garantendo la connettività finale.
2. **Fase Verticale:** Per ciascun set distinto isolato sulla riga corrente, l'algoritmo seleziona casualmente almeno una cella (e potenzialmente più di una) e ne abbatte il muro inferiore (Sud), proiettando la connettività del set sulla riga sottostante. Le celle della riga successiva che non ricevono una connessione verticale vengono inizializzate con un identificativo di set completamente nuovo e vergine.

4.5.4 Analisi della Complessità Teorica

L'aspetto più straordinario dell'algoritmo di Eller risiede nella sua efficienza computazionale e spaziale. Per elaborare una singola riga di larghezza W , l'algoritmo esegue scansioni

lineari sui vettori di stato dei set, impiegando un tempo costante per cella. Di conseguenza, l'intera griglia viene completata eseguendo un'unica passata fissa. La complessità temporale è strettamente lineare:

$$\mathcal{O}(V)$$

Dal punto di vista della memoria ausiliaria, l'algoritmo non necessita della conoscenza strutturale delle righe passate né di quelle future. Lo spazio richiesto è unicamente quello necessario a immagazzinare il vettore dei set della riga corrente, portando la complessità spaziale a un valore ottimale pari a $\mathcal{O}(\sqrt{V})$ (ovvero lineare rispetto alla sola larghezza della griglia W), rendendolo l'algoritmo ideale per la generazione di labirinti di dimensioni virtualmente infinite.

5 Esperimenti e Analisi Dati

5.1 Setup Sperimentale

Al fine di garantire la riproducibilità metodologica e di isolare l'efficienza algoritmica pura dalle interferenze di sistema, le misurazioni prestazionali sono state condotte all'interno di un ambiente hardware e software standardizzato, descritto nei seguenti punti:

- **Processore:** Architettura x86_64 CPU, con frequenza di clock stabilizzata per prevenire fenomeni di *thermal throttling*.
- **Memoria RAM:** Configurazione in Dual-Channel operante ad alta frequenza, con allocazione heap fissa iniziale e massima impostata sulla JVM tramite i flag `-Xms2G -Xmx2G` per azzerare i ritardi legati all'espansione dinamica della memoria.
- **Ambiente Software:** Java Run-Time Environment (JRE) versione 26, accoppiato al compilatore HotSpot Just-In-Time (JIT) ottimizzato al livello C2.

5.1.1 Metodologia di Profilazione e Campionamento

Come evidenziato nella letteratura scientifica legata al benchmarking su macchine virtuali regolate da Garbage Collection, la misurazione estemporanea del tempo di esecuzione (*wall-clock time*) produce dati affetti da un alto coefficiente di varianza statistica. Per neutralizzare tali anomalie architetturali, la raccolta dati è stata strutturata in due fasi sequenziali distinte:

1. **Verifica Strutturale Preventiva:** Ogni algoritmo esegue una generazione preliminare su una griglia di test. Un modulo di controllo analitico verifica che il grafo risultante costituisca un vero albero di copertura privo di partizioni sconnesse, validando l'equazione di Eulero per i grafi planari ($|E'| = |V| - 1$).
2. **Fase di Riscaldamento (Warm-up):** Ciascun algoritmo viene eseguito per 100 iterazioni consecutive non registrate. Questa procedura garantisce che le sezioni di codice più calde (*hot spots*) vengano intercettate dal profiler interno della JVM e tradotte direttamente in linguaggio macchina nativo altamente ottimizzato (loop unrolling, vettorializzazione hardware SIMD), stabilizzando i tempi prima della misurazione effettiva.

3. **Campionamento Iterativo:** La misurazione reale viene calcolata estraendo la media aritmetica su un blocco di 50 generazioni distinte per ogni singola taglia spaziale, utilizzando la funzione nativa di precisione `System.nanoTime()` convertita successivamente in scala millisecondi (ms/op).

5.2 Risultati Sperimentali Globali

Le istanze di input seguono una progressione spaziale quadrata definita dall'insieme di nodi V , con un lato della griglia $N \in \{10, 20, 30, 50, 100\}$, coprendo un dominio di calcolo che spazia da un minimo di 100 celle fino a un massimo di 10.000 celle complessive per labirinto.

Nella Tabella seguente sono racchiusi i dati empirici ricavati dall'esecuzione del benchmark headless:

Algoritmo	10x10	20x20	30x30	50x50	100x100
Randomized DFS	0,0096	0,0248	0,2396	0,2097	0,6800
Randomized Kruskal	0,0291	0,0522	0,1215	0,2410	1,2001
Randomized Prim	0,0194	0,0842	0,2172	0,5796	2,4104
Aldous-Broder	0,0237	0,1634	0,4405	1,2573	6,4577
Wilson's Algorithm	0,0289	0,2006	0,8743	5,2983	79,6242
Recursive Division	0,0132	0,0180	0,0306	0,0804	0,3833
Eller's Algorithm	0,0130	0,0600	0,1249	0,3668	2,0820

Tabella 1: Tempi medi di esecuzione espressi in millisecondi (ms/op) al variare delle dimensioni della griglia.

5.3 Discussione Critica dei Risultati

L'analisi comparativa dei dati campionati mette in evidenza una discrepanza sistematica e affascinante tra la complessità asintotica teorica descritta nel Capitolo 2 e il comportamento macroscopico degli algoritmi sulla memoria fisica della Java Virtual Machine. I sette algoritmi testati rivelano dinamiche di scaling radicalmente distinte, che permettono di classificarli in tre macro-regimi prestazionali.

5.3.1 Il dominio degli approcci Top-Down: Divisione Ricorsiva

I dati empirici decretano l'algoritmo di **Divisione Ricorsiva** come l'approccio in assoluto più efficiente sull'intero spettro di campionamento, registrando appena 0,3833 ms sulla taglia massima 100×100 ($|V| = 10.000$).

Sebbene la sua complessità teorica sia $\mathcal{O}(V \log V)$ — dunque asintoticamente superiore alla linearità pura di Eller o del DFS — questo algoritmo beneficia di un vantaggio strutturale cruciale legato all'architettura hardware:

- **Assenza di Pointer-Chasing:** A differenza degli algoritmi operanti sui grafi, la Divisione Ricorsiva non esegue traiettorie esplorative saltando da un riferimento di memoria all'altro (meccanica che causa continui *cache miss* nei livelli L1/L2).

- **Località spaziale e JIT:** L'algoritmo opera sezionando geometricamente blocchi contigui di matrici. Il compilatore JIT HotSpot intercetta la sequenzialità dei cicli di inserimento dei muri, applicando autonomamente ottimizzazioni di *loop unrolling* e delegando alla CPU la pre-inizializzazione dei blocchi in cache. Le costanti nascoste della complessità temporale risultano pertanto microscopiche.

5.3.2 L'efficienza dei Graph Traversal e delle Strutture Dati: DFS, Kruskal ed Eller

Nel secondo regime prestazionale si posizionano il **Randomized DFS** (0,6800 *ms*), **Kruskal** (1,2001 *ms*) ed **Eller** (2,0820 *ms*).

- **Randomized DFS:** Si conferma il migliore tra gli approcci orientati ai nodi. Operando in tempo linearmente puro $\mathcal{O}(V)$, l'uso di uno stack vettorializzato riduce l'overhead di allocazione al minimo. Il leggero flesso osservabile tra la taglia 30×30 e 50×50 è un comportamento tipico delle misurazioni JVM, imputabile al superamento della soglia di compilazione del JIT che converte la funzione in codice nativo stabilizzato (steady-state).
- **Kruskal Modificato:** Mostra uno scaling straordinariamente controllato grazie all'efficienza ammortizzata del Disjoint-Set. La compressione dei cammini azzerava quasi completamente il costo delle unioni, rendendo l'inizializzazione stocastica (`Collections.shuffle`) l'unico vero fattore limitante su larga scala.
- **Eller's Algorithm:** Pur registrando tempi leggermente superiori a Kruskal a 100×100 , mantiene un andamento strettamente lineare. L'overhead residuo è legato unicamente alla gestione dinamica delle tabelle associative di hashing (`java.util.Map`) necessarie per raggruppare i set durante la fase di proiezione verticale riga per riga.

5.3.3 Analisi del collasso asintotico nei Random Walk: Wilson vs Aldous-Broder

Il dato indubbiamente più rilevante ed eccentrico dell'intero benchmark risiede nel comportamento dell'algoritmo di **Wilson**, il quale collassa verticalmente sulla taglia 100×100 raggiungendo ben 79,6242 *ms*, distaccando l'altrimenti inefficiente **Aldous-Broder** (6,4577 *ms*) di oltre un ordine di grandezza.

Questo fenomeno rappresenta un perfetto esempio di scontro tra l'eleganza matematica astratta e il vincolo implementativo delle strutture dati:

1. **L'illusione dell'efficienza asintotica:** Sulla carta, l'algoritmo di Wilson schiaccia Aldous-Broder poiché la rimozione dei cicli impedisce alla passeggiata casuale di perdersi nel grafo.
2. **Il collo di bottiglia di `ArrayList.indexOf()`:** Per identificare la formazione di un ciclo all'interno della passeggiata corrente, l'implementazione esegue l'istruzione `walkPath.indexOf(next)`. La struttura `ArrayList` effettua questa ricerca tramite una scansione lineare sequenziale, con costo pari a $\mathcal{O}(L)$, dove L è la lunghezza attuale del cammino.

3. **Degradazione nel regime macroscopico:** Quando la griglia scala a 10.000 celle, nelle fasi iniziali in cui l'albero consolidato è ancora molto piccolo, la passeggiata stocastica copre distanze enormi nel vuoto prima di intersecare l'albero. Il vettore `walkPath` si estende per migliaia di elementi; di conseguenza, ogni singolo passo della passeggiata casuale non costa più $\mathcal{O}(1)$, ma decade in un costo lineare $\mathcal{O}(L)$ dovuto al controllo dei cicli.

Al contrario, **Aldous-Broder**, pur eseguendo un numero di passi totali statisticamente molto più elevato a causa del *Coupon Collector's Problem*, esegue ad ogni singolo *step* un controllo atomico in tempo costante $\mathcal{O}(1)$ (verificando semplicemente il flag booleano `visited` sulla matrice). Di conseguenza, su domini inferiori alle 10.000 celle, l'assenza di sovrastrutture di ricerca rende Aldous-Broder drasticamente più veloce di Wilson, ribaltando il verdetto della teoria asintotica pura.

5.3.4 Andamento dell'algoritmo di Prim

L'algoritmo di **Prim Modificato** si attesta a 2,4104 *ms* sulla taglia massima. L'andamento evidenzia chiaramente la natura quadratica $\mathcal{O}(V^2)$ della nostra implementazione a frontiera. Man mano che il labirinto si espande, la dimensione della lista dei muri candidati cresce proporzionalmente al perimetro dell'albero; l'operazione di rimozione casuale dall'array introduce quindi uno slittamento di memoria che ne frena le prestazioni rispetto a Kruskal, confermandone i limiti strutturali descritti in fase teorica.

Riferimenti bibliografici

- [1] Wikipedia contributors, “Maze generation algorithm,” *Wikipedia, The Free Encyclopedia*, https://en.wikipedia.org/wiki/Maze_generation_algorithm.